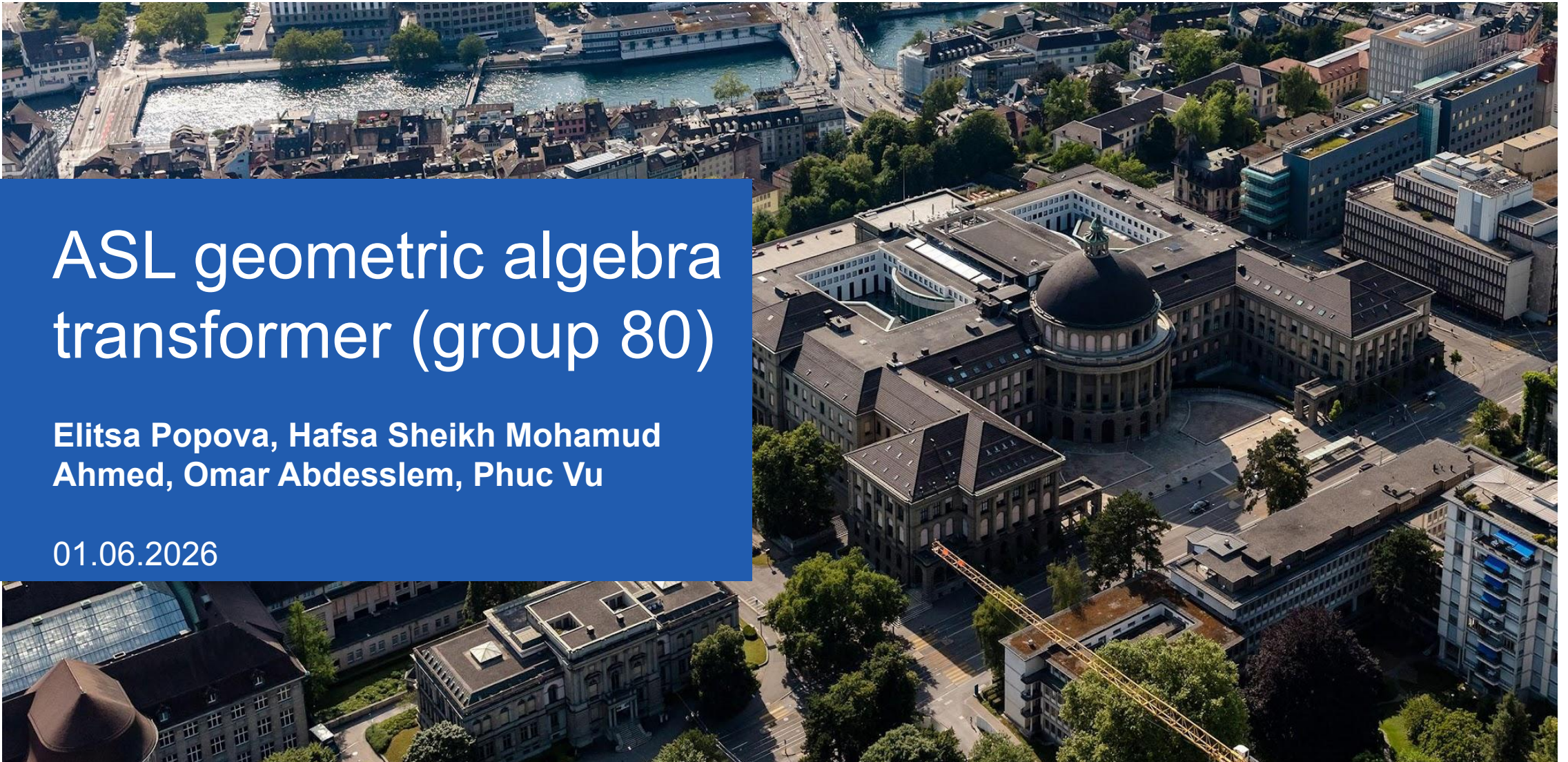


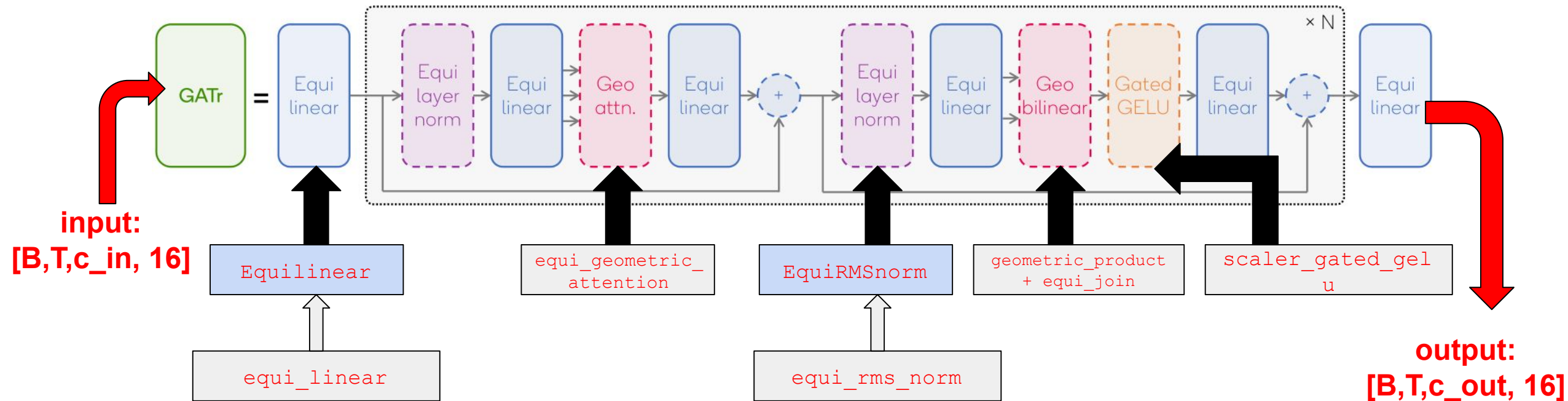
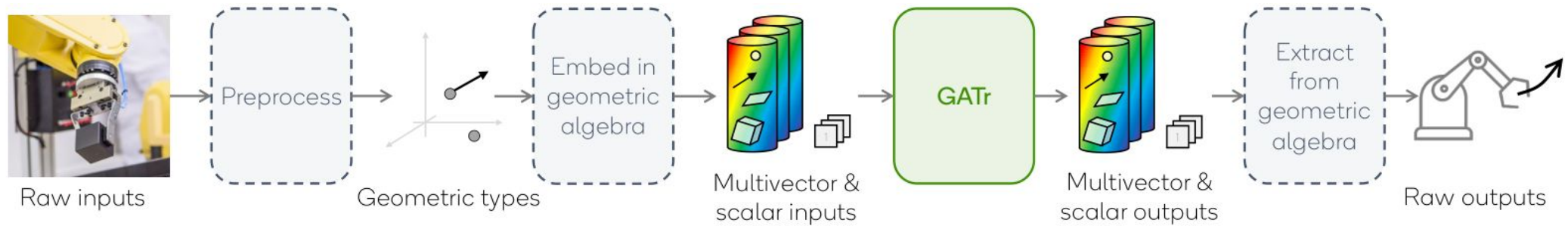
ASL geometric algebra transformer (group 80)

Elitsa Popova, Hafsa Sheikh Mohamud
Ahmed, Omar Abdesslem, Phuc Vu

01.06.2026



GATr algorithm overview



GATr algorithm overview

equi_linear $O(B T C_{\text{in}} C_{\text{out}} \cdot 9 \cdot 16^2)$

Input: $x \in \mathbb{R}^{B \times T \times C_{\text{in}} \times 16}$, $\text{weight} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 9}$, $\text{basis} \in \mathbb{R}^{9 \times 16 \times 16}$

Output: $y \in \mathbb{R}^{B \times T \times C_{\text{out}} \times 16}$

- 1: **for** each token (p, t) , each $o \in [C_{\text{out}}]$, each $d \in [16]$ **do**
- 2: $y[p, t, o, d] \leftarrow 0$
- 3: **for** $i \in [C_{\text{in}}]$, $w \in [9]$, $s \in [16]$ **do**
- 4: $y[p, t, o, d] += \text{weight}[o, i, w] \cdot \text{basis}[w, d, s] \cdot x[p, t, i, s]$

geometric_attention $O(B H T^2 C)$

Input: $Q, K, V \in \mathbb{R}^{B \times H \times T \times C \times 16}$

Output: $O \in \mathbb{R}^{B \times H \times T \times C \times 16}$

- 1: $\tilde{q}, \tilde{k} \leftarrow$ select IPA blades $\{0, 2, 3, 4, 8, 9, 10, 14\}$ and form DAA terms via $\text{basis}[4, 4, 5]$
- 2: **for** each (b, h) **do**
- 3: **for** $i \in [T]$, $j \in [T]$ **do**
- 4: $S[i, j] \leftarrow \frac{1}{\sqrt{12C}} \sum_c (\langle \tilde{q}_{i,c}, \tilde{k}_{j,c} \rangle_{\text{ipa}} + \langle \cdot \rangle_{\text{daa}})$
- 5: $A \leftarrow \text{softmax}_{\text{row}}(S)$; $O_{b,h} \leftarrow A V_{b,h}$

equi_join $O(B T C \cdot 16^3)$

Input: $x, y \in \mathbb{R}^{\cdots \times 16}$, $\text{join_kernel} \in \mathbb{R}^{16 \times 16 \times 16}$

Output: $z \in \mathbb{R}^{\cdots \times 16}$

- 1: **for** each row, each output blade $i \in [16]$ **do**
- 2: $z[i] \leftarrow \sum_{j \in [16]} \sum_{k \in [16]} \text{join_kernel}[i, j, k] \cdot x[j] \cdot y[k]$
- 3: (optional) $z *= \text{reference}[\dots, 14]$

geometric_product $O(B T C \cdot 16^3)$

Input: $x, y \in \mathbb{R}^{\cdots \times 16}$, $\text{cayley} \in \mathbb{R}^{16 \times 16 \times 16}$

Output: $z \in \mathbb{R}^{\cdots \times 16}$

- 1: **for** each row, each output blade $i \in [16]$ **do**
- 2: $z[i] \leftarrow 0$
- 3: **for** $j \in [16]$, $k \in [16]$ **do**
- 4: $z[i] += \text{cayley}[i, j, k] \cdot x[j] \cdot y[k]$

$$O(L B T (C^2 + H T C)) = O(L B (T C^2 + H T^2 C))$$

Cost analysis

- Counted by hand (using script) from the C++ kernel source (inner-loop op counts × loop bounds), implemented as a parametric model in `cpp_flop_counter.py`
- Two versions reported:
 - `flops_total`: every op counts as 1
 - `weighted_flops`: non-trivial ops cost more: `div=4`, `sqrt=8`, `exp=20`, `tanh=20` (`add/mul/pow2 = 1`), reflecting real latency on the target CPU

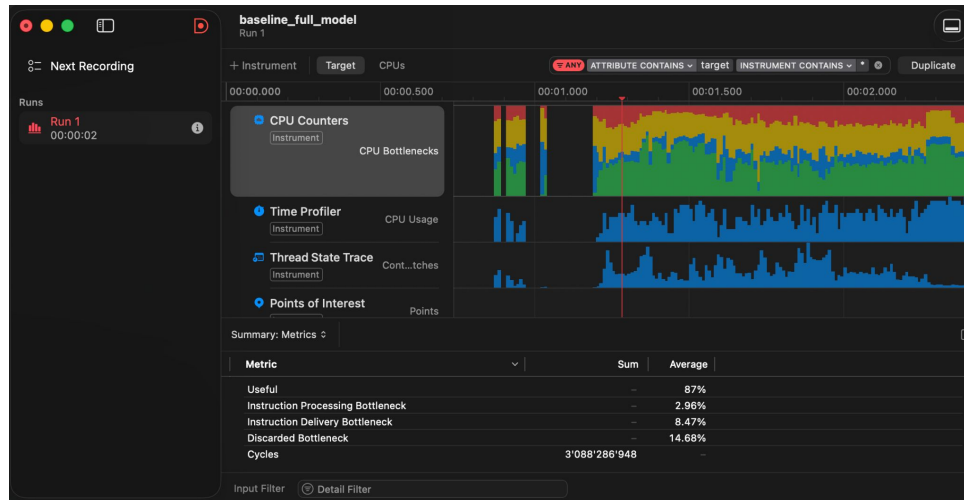
Formulas for baseline:

- **EquiLinear** : $2 \cdot B \cdot T \cdot C_{\text{out}} \cdot 16 \cdot C_{\text{in}} \cdot 9 \cdot 16$ muls + half that value of adds
- **GeometricProduct**: $m \cdot 16^3$ terms, 2 mul + 1 add each
- **EquiJoin**: $m \cdot 16 \cdot 16 \cdot 16$ terms, 2 mul + 1 add each
- **GeometricAttention**: score dot ($B \cdot H \cdot T_q \cdot T_k \cdot D$) + softmax (exp, div) + value accumulation ($T_k \cdot D_v$) + daa transform gives 165 mul/81 add/1 div per token
- **EquiRMSNorm**: 8 mul + add per channel (inner product) + 1 div + 1 sqrt per row + 16 muls rescale
- **ScalarGatedGELU**: 2 mul + 2 add + 1 tanh per element

Profiling

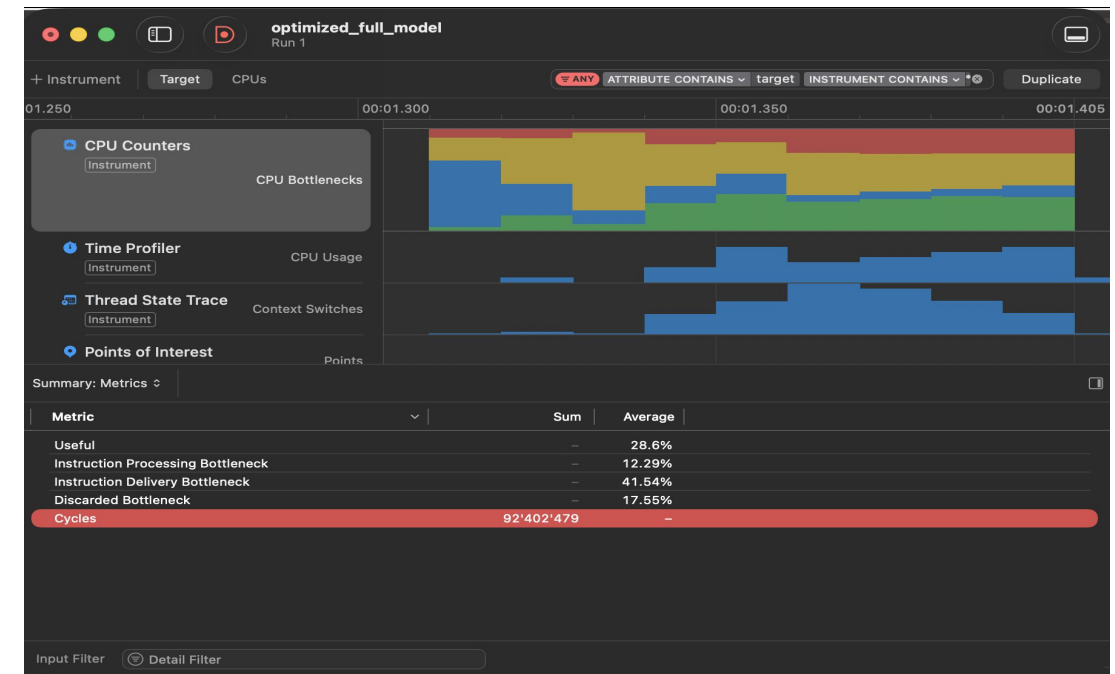
- We used the Instruments App on Apple Silicon on XCode
- Ran it 100 times
- Outputted the log and .trace

Profiling: Instruction Processing Bottleneck

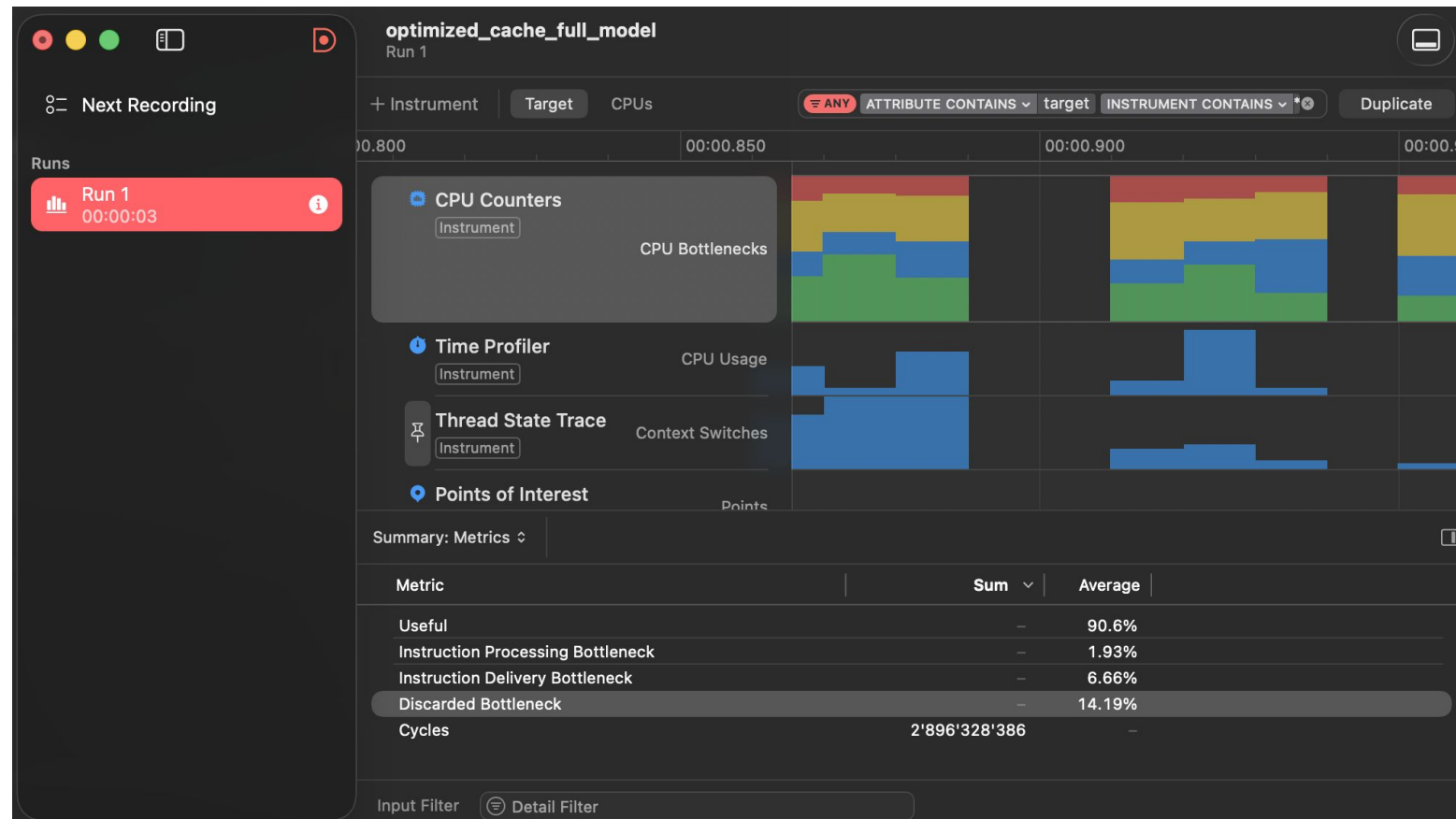


- (1) For Baseline, Instruments does not identify Bottlenecks, just recommends algorithm improvement as Useful

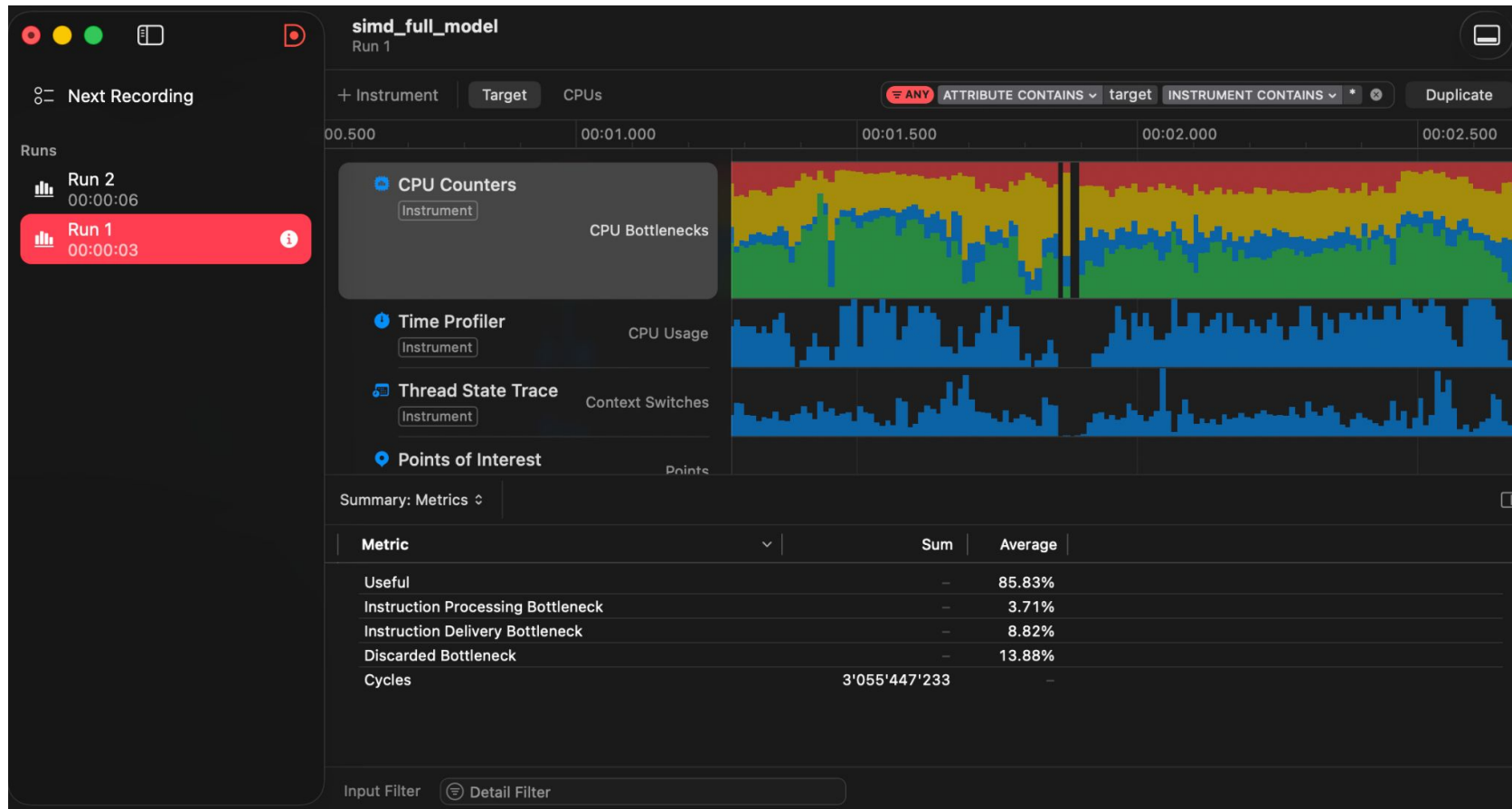
- (1) After CPU optimizations (our manual hardcodings), Instruments identifies Instruction Delivery Bottlenecks and Instruction Processing bottlenecks



Profiling: Optimized for cache Model shows high optimization



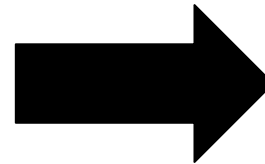
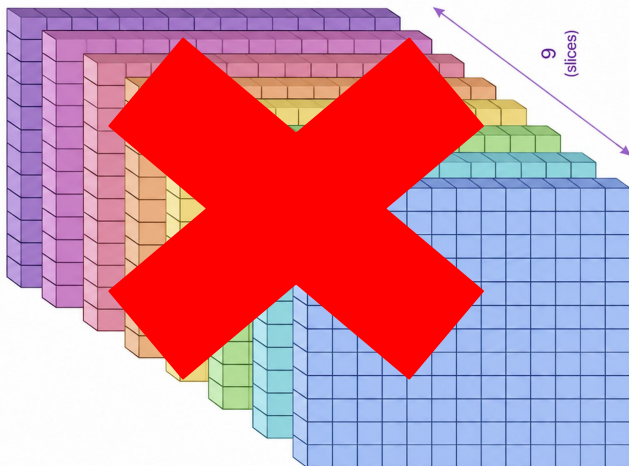
Profiling: Final SIMD Model shows high optimization



Optimizations

equi_linear :

- sparsity exploitation (hardcode the 24 non-zero positions instead of looping over all $9 \times 16 \times 16$ entries)
 - => avoids unnecessary computation + basis tensor loads
 - accumulators
 - move out pointer arithmetic out of inner loops
- after normalization every non-zero of the basis is one of 4 values, move basis multiplication out of inner loop



```
out_o[0] = a0;
out_o[1] = a1_diag + a1_off;
out_o[2] = a2;
out_o[3] = a3;
out_o[4] = a4;
out_o[5] = a5_diag + a5_off;
out_o[6] = a6_diag + a6_off;
out_o[7] = a7_diag + a7_off;
out_o[8] = a8;
out_o[9] = a9;
out_o[10] = a10;
out_o[11] = a11_diag + a11_off;
out_o[12] = a12_diag + a12_off;
out_o[13] = a13_diag + a13_off;
out_o[14] = a14;
out_o[15] = a15_diag + a15_off;
```

Optimizations

equi_linear :

- for cache locality: reorganized (transposed) the weights in equi_linear to fit the cache size, increasing spatial locality (restructuring data into SoA)

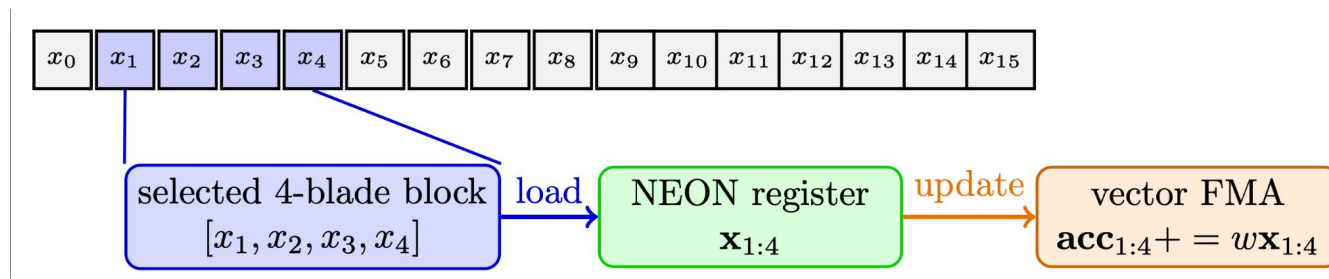
[channels_in, channels_out, 9] => [channels_out, channels_in, 9]

Optimizations

equi_linear :

Vectorization:

- Vectorize across blades: one output channel at a time.
- 4 NEON accumulators for the contiguous blade groups (1–4, 5–8, 11–14)
- 9 scalar accumulators for the rest (blade 0, 9, 10, 15, and the off-diagonal grade-lowering terms)



Optimizations

geometric_product:

first optimizations:

- We used generated kernels, hardcoding the sparsity instead of using nested loops.
- We also applied more optimizations, to further bring down the runtime like using FMAs and reducing redundant recomputations

Next optimizations, vectorization:

- First transposes the input from the natural AoS layout into SoA
- Runs the same number of operations but now each instruction does 4 batches in parallel using `vfmaq_f32`.
- Transposes the result back.

Optimizations

`geometric_product:`

`gp kernel after optimizations:`

```
int64_t b = 0;
for (; b + 4 <= batch_size; b += 4) {
    const int64_t b16 = b * 16;
    gp_kernel_four_batch(x_ptr + b16, y_ptr + b16, result_ptr + b16);
}
```

Example of one output blade
computation (after SIMD
optimization):

```
o[0] = vmulq_f32(x[0], y[0]);
o[0] = vfmaq_f32(o[0], x[2], y[2]);
o[0] = vfmaq_f32(o[0], x[3], y[3]);
o[0] = vfmaq_f32(o[0], x[4], y[4]);
o[0] = vfmsq_f32(o[0], x[8], y[8]);
o[0] = vfmsq_f32(o[0], x[9], y[9]);
o[0] = vfmsq_f32(o[0], x[10], y[10]);
o[0] = vfmsq_f32(o[0], x[14], y[14]);
```

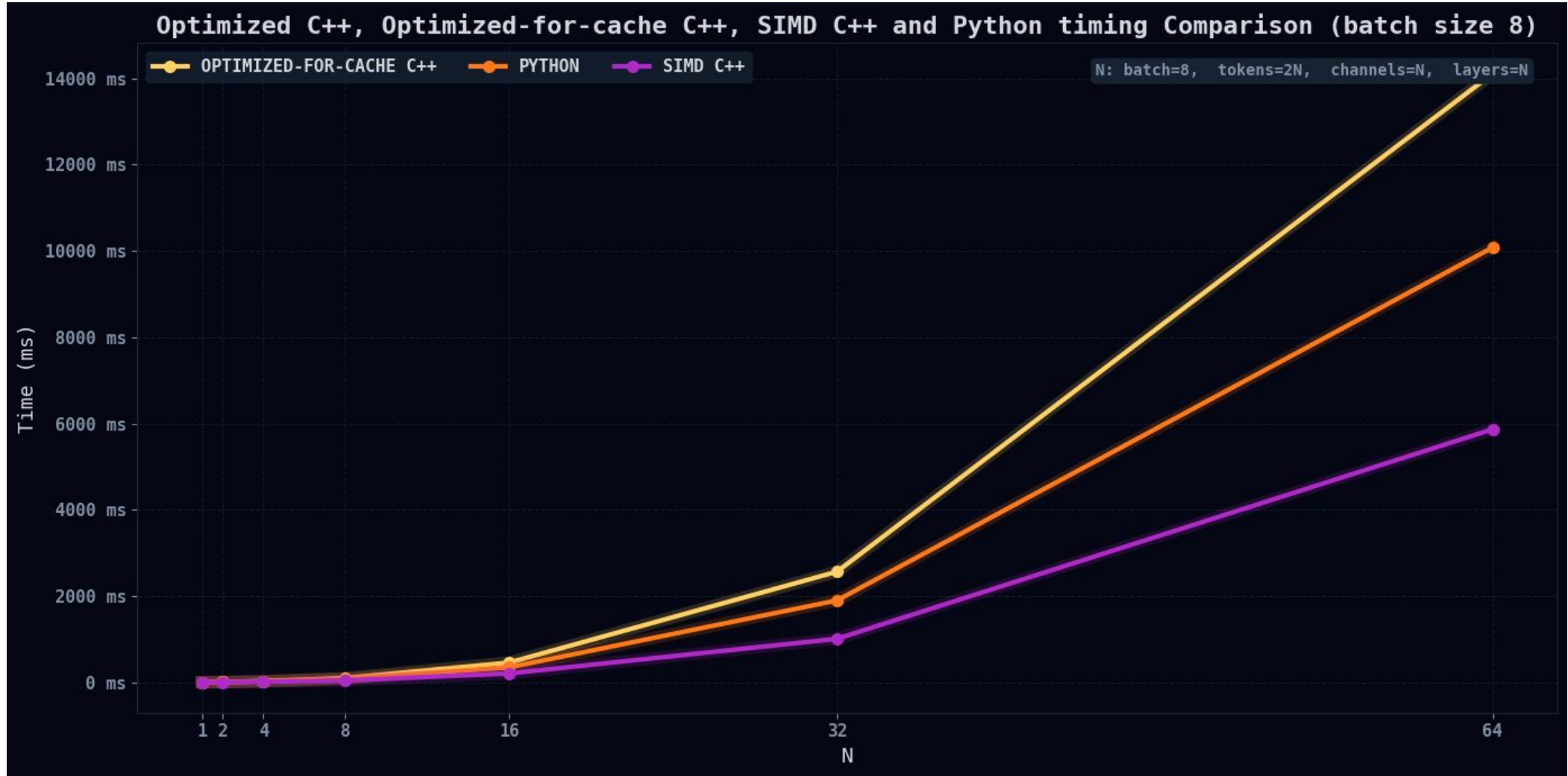
Timing results comparison: all



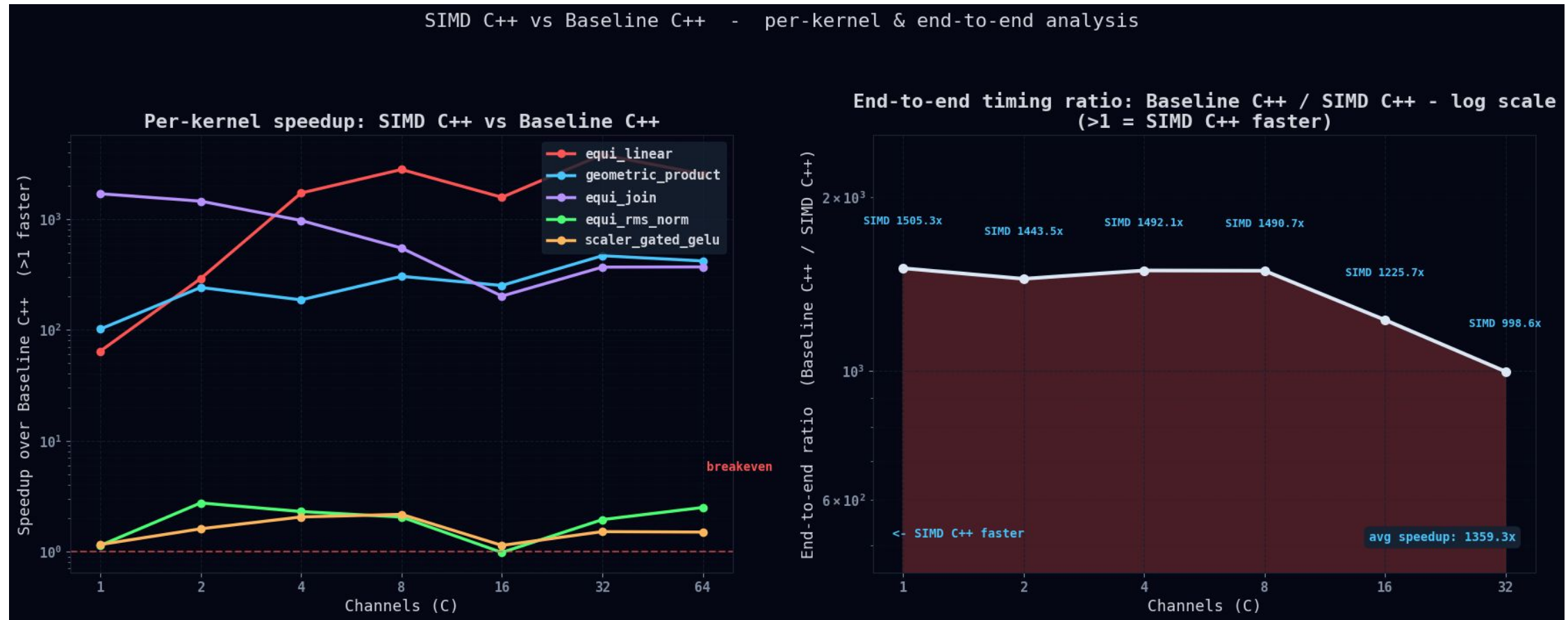
Timing results comparison: all, log-scale



Timing Results after vectorization

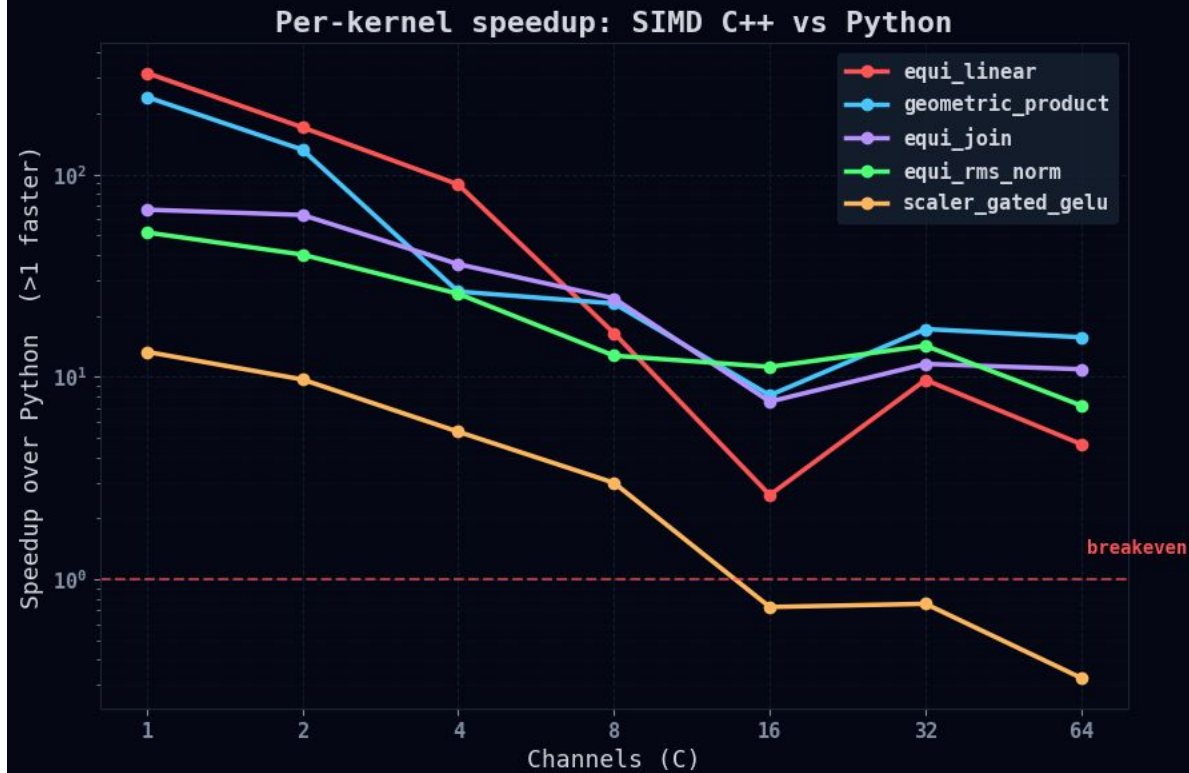


Per function timing analysis: vs Baseline

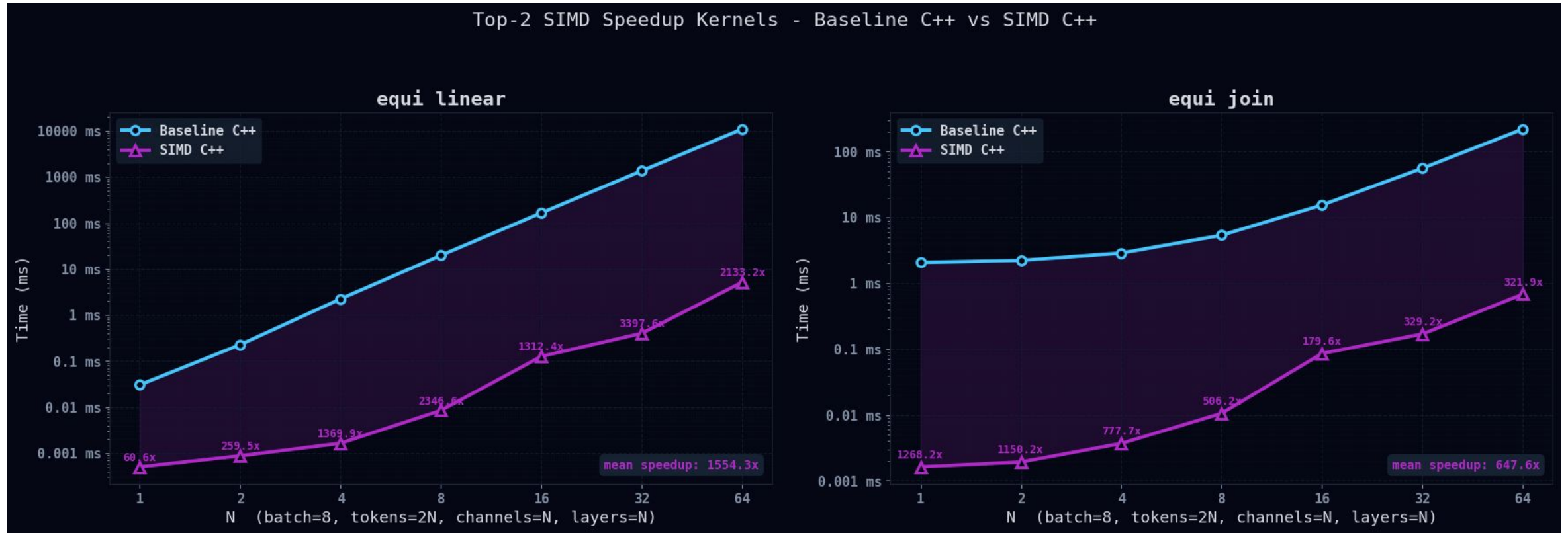


Per function timing analysis: vs Python

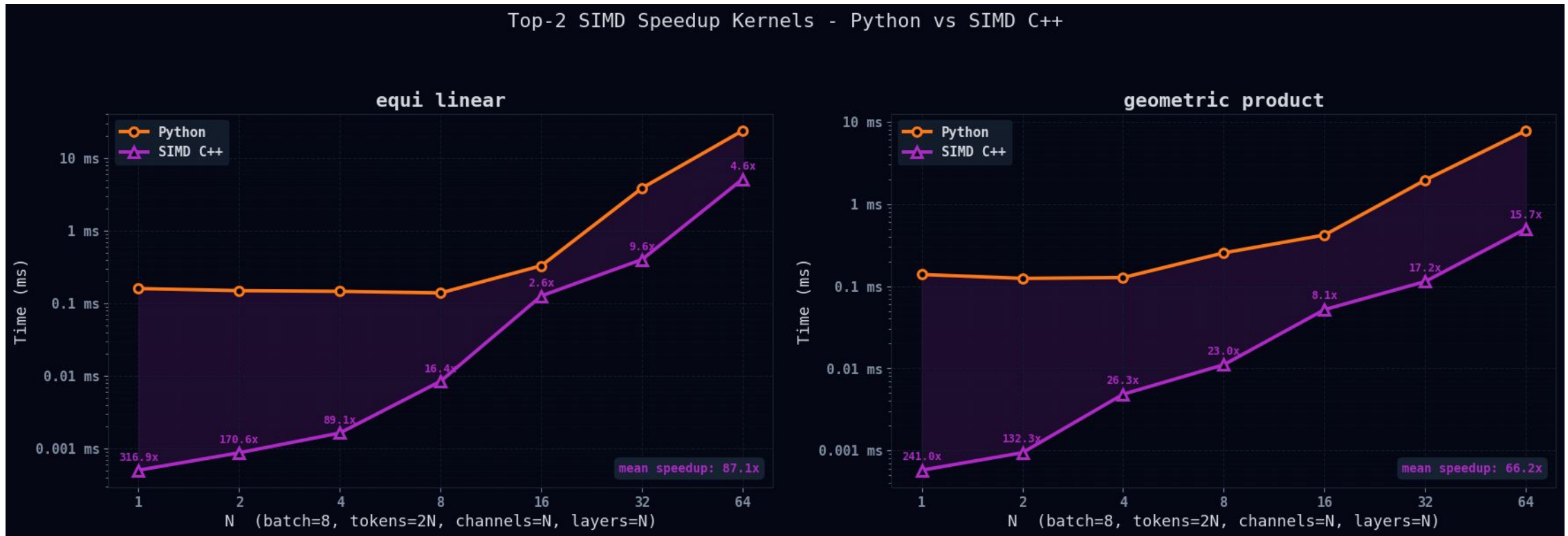
SIMD C++ vs Python - per-kernel & end-to-end analysis



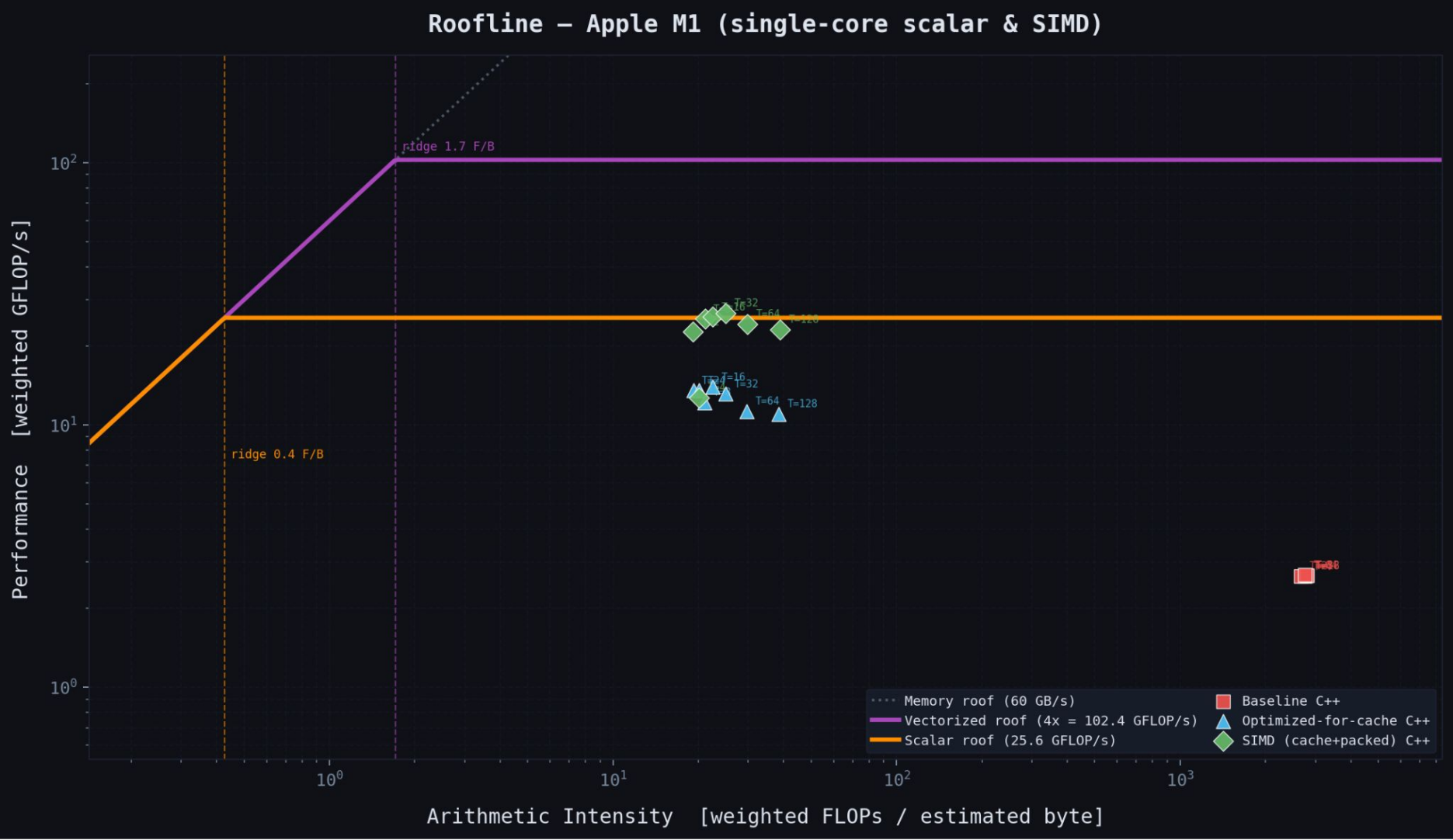
Top 2 speed-ups: SIMD vs Baseline



Top 2 speed-ups: SIMD vs Python



Roofline Plot



Thank you for listening !